Regression based extraction of apple tree skeletons

MMA3001 Numerical Methods and Machine Learning
Dataset 1 individual project

Nelson Bawden: Student ID 34976698: GitHub repository: https://github.com/Nelson-Bawden/MMA3001-Invdividual-Project-34976698

1 Engineering problem and scope

The specific question is whether an interpretable background-regression model can reduce irrelevant image structure while retaining the visible tree trunk and branches for tree identification. The intended output is an image-coordinate representation that an engineer can inspect and compare across observations.

The scope is restricted to paired, already aligned RGB and grayscale depth observations of individual trees. Depth values are treated as recorded intensities because their physical encoding has not been established. Camera calibration parameters and a verified conversion from depth intensity to physical distance were not available for this work. I therefore restricted the implementation to two-dimensional image skeletons rather than presenting an assumed geometry as an accurate three-dimensional reconstruction. The code does not align viewpoints, recover hidden branches or match trees between seasons. This is a narrower contribution to the dataset's broader reconstruction objective, and acceptance of the reduced scope remains a matter for the teaching team.

2 Data and input output definitions

Dataset 1 contains RGB and depth observations from multiple trees and seasons (summer and winter) [2]. The local collection includes Dataset and Dataset (With Summer GT). The latter is not automatically treated as ground truth: its annotation format and correspondence must first be verified. The current evaluation uses two complete uploaded pairs.

Item	Accepted representation and meaning
RGB input	PNG, TIF or TIFF; converted to three colour channels in [0, 1]. Pixel coordinates represent image position.
Depth input	Same pixel dimensions as RGB; grayscale or three equal channels.
Observation identity	Tree ID, season and capture time parsed from filenames, e.g. R10N07_Winter_D-12-10-12.png.
Main outputs	Comparison PNGs; per-method results.csv; regression_models.csv; sensitivity.csv; summary.csv; settings.json and checks.txt.

Pairing and invalid data

The code recursively searches the selected folder. A pair is accepted only when exactly one RGB and one depth file share tree, season and time identity data. Unmatched or ambiguous captures are skipped. Image-size mismatches and colorized depth images are rejected; missing-depth pixels become NaN.

Processing failures caught as ValueError or OSError are recorded in errors.csv when present. No valid pairs cause the run to stop to save time and cut back on repeating excessive matches. The aerial image supplies context but is not used for localisation or measurement.

3 Linear regression model

The background model is intentionally small and interpretable. It assumes that rows with many valid depth pixels often contain a broad background surface, and that their median intensity changes approximately linearly with image height. Row medians reduce the influence of a narrow trunk only when background occupies most valid pixels in that row because of the ground surface creating noise that makes it harder to distinguish the trunk from the terrain. The model is fitted independently to each observation.

Background samples and least squares

Let D(u, v) denote raw depth intensity at column u and row v, and V(u, v) indicate finite depth. For width W, a row is dense when its valid fraction exceeds 0.40, which is used as a heuristic for a broad background surface rather than proof of terrain. The row coordinate is normalised using the full image height H to keep its numerical scale moderate.

$$y(v) = 2v / (H - 1) - 1 \quad and \quad B(v) = b_0 + b_1 y(v)$$

For n dense rows, each row of the design matrix A is [1, y(v)], and the target vector m contains their medians. Each sampled row receives equal weight. The fitted coefficients minimise the total squared residual:

$$b* = arg\,min \|Ab - m\|_2{}^2$$

The implementation calls numpy.linalg.lstsq(A, m, rcond=None), which computes a least-squares solution and returns the matrix rank [3]. It checks for rank two rather than explicitly forming an inverse. However, this does not remove bias from incorrectly selected background samples.

Foreground decision

Let q_5 and q_{95} be the fifth and ninety-fifth percentiles of all valid depth intensities. The threshold is $\tau = c(q_{95} - q_5)$, with c = 0.10 initially. On dense rows, a finite pixel is retained when the signed difference from the predicted background exceeds τ:

$$s[D(u,v) - B(v)] > \tau \quad with \quad s = +1$$

The direction s is configurable because display intensities may increase or decrease with depth. Its current sign is an empirical setting, not a statement that brighter always means nearer. Sparse rows retain valid pixels without background subtraction. If fewer than three dense rows exist, the whole observation uses a recorded valid-depth fallback. Therefore, regression is central to background removal where it is supported, but is not applied to every retained pixel.

regression_models.csv records the coefficients, number and fraction of dense rows, fitted residual RMSE, status and fraction of valid pixels in sparse rows. RMSE describes agreement with the sampled row medians; it cannot establish whether retained pixels are a part of the main trunk.

4 Segmentation skeleton extraction and alternatives

Connectivity and skeleton extraction

After depth thresholding, one iteration of binary closing joins small gaps, although it can also join unrelated foreground regions. The default two-dimensional structure is cross-shaped. Eight-connected components are then labelled. The component with the greatest overlap with a lower-centre seed rectangle is retained. This rectangle spans 35–65% of image width and 55–88% of image height. No overlap produces an empty mask rather than selection of an unrelated component.

The selected mask is thinned with a NumPy implementation of the two-sub step Zhang–Suen method [4]. Iteration stops when neither substep deletes a pixel. Because pixels are only removed from a finite image, the process terminates. The result is an image centerline.

Baseline and design justification

The baseline retains pixels with excess-green value $2G - R - B$ below 0.08 and mean RGB brightness between 0.03 and 0.72, then applies the same thinning routine. It represents an easy method that visible wood is relatively dark and less green than foliage. Grass, shadows and supports can satisfy this rule. The baseline uses no depth-background model and no trunk-component selection.

Approach	Reason to consider it	Principal trade-off
Colour baseline	Simple, transparent and independent of depth availability.	Sensitive to lighting and background colour.
Linear background regression	Two interpretable fitted parameters	Assumes a row-dependent linear trend and background-dominated samples.

Implementation organization

One script contains pairing/loading, regression/segmentation, thinning, evaluation and the main run loop. Two folder-selection dialogs choose inputs and results. Each run creates a timestamped directory, and all comparison images share one comparisons folder.

5 Verification and validation design

Completed implementation verification

Check	Criterion	Observed outcome
Regression coefficients	Recover $b_0 = 30$ and $b_1 = 10$ within numerical tolerance.	Passed
Synthetic foreground	Foreground IoU greater than 0.99 for the idealised trunk.	Passed
All missing depth	Recorded insufficient-data status and empty foreground.	Passed

Check	Criterion	Observed outcome
Thinning behaviour	One connected component, fewer pixels, unchanged by re-thinning.	Passed
Metric sanity	Identical prediction/reference gives IoU = 1 and F1 = 1.	Passed

These tests establish limited implementation correctness under controlled conditions. They do not prove topology preservation for every shape, robustness to realistic noise, or accuracy on foliage.

Required real-image validation

Validation asks whether the representation corresponds to actual visible wood. First inspect the annotation format in the supplied Summer GT collection. The current reader accepts binary masks, not spreadsheet skeleton coordinates, coloured overlays or depth images.

Where suitable masks are absent, manually annotate a manageable sample spanning winter/summer, depth loss, background clutter and off-centre trees. Define visible wood consistently and mark uncertain or obscured regions. Keep all observations of each physical tree in one split. Use development trees for thresholds and reserve different tree IDs for final testing.

6 Preliminary results and comparison

The current numerical evidence comes from one execution of two complete 640 × 480 pairs. Both are development observations. The script produced four comparison figures and no processing failure for these pairs.

Observation and method	Skeleton pixels	Depth coverage	Compute time s
Winter R10N07 baseline	47,927	42.85%	0.0517
Winter R10N07 regression	1,735	98.21%	0.0506
Summer R21U04 baseline	33,270	86.35%	0.1000
Summer R21U04 regression	3,039	100.00%	0.1248

Depth coverage is the fraction of skeleton pixels with finite input depth. High coverage is partly expected because the regression pipeline begins from valid depth pixels. It is not independent accuracy evidence. Likewise, fewer skeleton pixels may indicate successful background rejection, loss of true branches, or both. The observed counts alone do not distinguish these explanations.

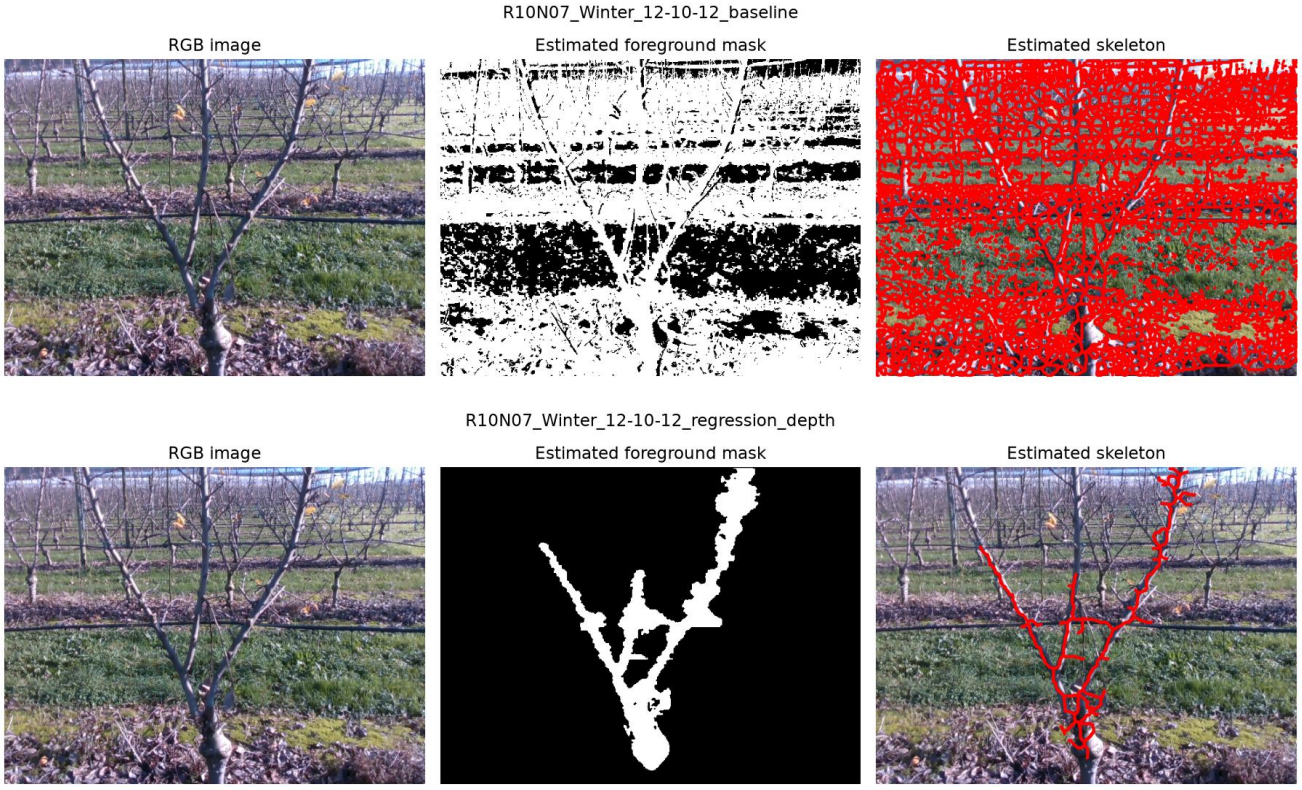


Figure 2. R10N07 winter comparison at c = 0.10. Identify retained trunk, missed branches and false foreground using the RGB image and reference annotation.

7 Performance evaluation interpretation and limitations

Timing and scalability

The main per-method timer includes mask extraction, fitting where applicable and thinning, but excludes loading, ground-truth evaluation and figure saving. On the two examples, regression took approximately 0.051 s and 0.125 s, versus 0.052 s and 0.100 s for the baseline. These are single-run measurements from the assistant's execution environment and do not establish end-to-end performance on my computer.

Working image storage scales with pixels. For a 640 × 480 image, float64 RGB alone uses about 7.37 MB and float64 depth about 2.46 MB, excluding masks, labels, temporary arrays and plots.

The implementation processes one pair at a time, closes figures after saving and reuses the regression fit during sensitivity analysis. These choices limit working memory and avoid repeated fitting, although their speed-up has not been measured separately. Rewriting accumulated results after each pair preserves progress but adds file-writing overhead. Saving two figures per observation also increases processing time and storage; its contribution should be measured before claiming an improvement in full-dataset throughput.

8 Conclusions

Main findings and engineering value

This project developed a single Python workflow that pairs RGB and depth observations, estimates a background-intensity trend using least-squares linear regression and extracts a two-dimensional skeleton from the selected foreground. The colour-only baseline supplies a simple reference against which the complete

regression process can be compared. The engineering contribution is an initial method for isolating visible tree structure for later identification work, rather than a complete tree-identification or navigation system.

The preliminary two-image execution showed that both pipelines ran successfully and that regression produced substantially smaller skeletons: 1,735 rather than 47,927 pixels in the winter example, and 3,039 rather than 33,270 in the summer example. These reductions demonstrate a change in the selected structure but do not, on their own prove that the retained structure is more accurate. Some removed pixels may be unwanted background and others may be genuine branches.

Justification for the two-dimensional scope

I did not pursue calibrated three-dimensional reconstruction because the camera intrinsics and verified depth scale were unavailable (at least as far as I could find). Without this information, a three-dimensional plot could be generated using assumed values, but its shape and dimensions could not be validated as a metric reconstruction such as depth of branches. A top-down photograph alone would not resolve these calibration and depth-scale uncertainties. The decision was therefore to produce a clearly defined image-coordinate output rather than imply accuracy that the available information could not support.

This restriction makes the method easier to interpret and test, but it also limits its engineering use. The skeleton cannot provide branch clearance in meters, recover hidden branches or establish a collision-free robot path.

Summer observations and seasonal comparison

Summer images are more difficult to use for visible branch extraction because leaves and apples obscure portions of the woody structure and introduce additional colour, depth and silhouette features. These features are real parts of the scene, but they act as clutter relative to the objective of extracting branches. They can interrupt visible branch paths, dominate row-median background samples or remain in the foreground mask. Thinning can then produce lines through clusters of leaves and fruit that should not be interpreted as actual branches.

For this branch-based approach, summer observations are therefore less directly useful for comparing tree structure than observations in which the branches are exposed. A summer skeleton may describe the visible canopy more strongly than the underlying wood, making direct comparison with a winter skeleton potentially misleading. This also limits its use as a structural signature for identifying the same tree across seasons.

Lessons and next steps

Overall, the project provides a reproducible computational starting point and a defensible explanation of its assumptions. Its strongest supported outcomes are successful execution, controlled verification and an interpretable regression-based extraction method. Further development should address the demonstrated failure cases and obtain calibration information before extending the output to physical three-dimensional geometry.

9 AI use reflection and references

Declaration of assistance

I used ChatGPT through Codex as a learning and development assistant [6]. Its contribution was substantial to conceptual understanding of the thinning algorithms: it drafted and modified the majority of the Python code, helped explain methods and errors, suggested checks, executed limited tests in its own environment to ensure that I wasn't running the whole data set with an error. It also explained the Zhang–Suen thinning method,

which I was unfamiliar with at the start of the project; its computational advantage over alternatives was not established by a controlled benchmark. But ChatGPT was able to guide me through understanding this and how to implement it. I directed the scope of the project and the type of computational approach that I wanted to use. AI was used to explain unfamiliar steps and reduce repetitive implementation work; its material contribution is acknowledged here.

AI explanations supported the distinction between file pairing and image registration, verification and validation, and raw depth intensity and physical distance. I remain responsible for the submitted claims, references, data handling and final engineering decisions. AI also assisted with drafting and revising this report, including its structure and conclusions.

Examples of AI assistance

The following prompt examples are edited summaries of the discussion, not verbatim transcript quotations. The responses summarise the assistance received.

Prompt summary	**Response summary and relevance**
so does this create the 2d skeleton on the linear regression method I specified earlier by comparing the depth image and the rgb image and then matches them up?	Explained filename pairing, separate extraction methods and the assumption of existing pixel alignment. This clarifies what the code actually does.
Based on the code so far are there any key areas that need to be expanded upon past the list I gave earlier.	Identified missing ground-truth validation, tree-level splitting, interpretation.

References

[1] Monash University. (2026). MMA3001 Numerical Methods and Machine Learning project brief. Supplied unit document, especially pp. 4–8 and assessment rubric.

[2] Monash University. (2026). MMA3001 project datasets. Dataset 1 RGB-D Apple-Tree Reconstruction and Orchard Mapping. Supplied dataset brief, p. 1.

[3] NumPy Developers. (n.d.). numpy.linalg.lstsq. NumPy reference documentation. https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html (accessed 25 October 2026).

[4] Zhang, T. Y., & Suen, C. Y. (1984). A fast parallel algorithm for thinning digital patterns. Communications of the ACM, 27(3), 236–239. https://doi.org/10.1145/357994.358023

[5] Monash University. (2026). MMA3001 unit topic schedule. Supplied screenshots identifying Topic 5.1 Linear Regression.

[6] OpenAI. (2026). ChatGPT with Codex. AI-assisted project conversation, code development and report drafting; model version not independently recorded.